

# Documentação Técnica

Gateway de Pagamento com Roteamento Inteligente Multi-Adquirente

1. Visão Geral
2. Arquitetura & Stack
3. Autenticação & Autorização
4. Motor de Roteamento
5. Motor Antifraude
6. Transações
7. Subadquirentes
8. Integração — Necta Multi-Pay
9. Webhooks
10. Conciliação
11. Configuração & Variáveis de Ambiente
12. Testes Automatizados
13. Deploy & Infraestrutura
14. Histórico de Versões
15. Pendências Conhecidas

---

**Versão do documento:** referente à versão de app **v1.3.1**

**Empresa:** COMPLYSOFT DESENVOLVIMENTO DE SISTEMAS LTDA

**Data de geração:** 11 de agosto de 2026

# 1. Visão Geral

---

O **ComplyRoute** é um gateway de pagamento que atua como camada de orquestração entre o lojista/marketplace e múltiplos subadquirentes (Cielo, Rede, Stone, PagSeguro, GetNet, SafraPay, Zoop, Asaas, OrendaPay e Necta Multi-Pay). Ele decide, transação a transação, para qual subadquirente enviar o pagamento — por método, valor, parcelamento, país do cartão, score de risco e saúde operacional dos próprios subadquirentes — e expõe uma API única e estável para quem consome a plataforma, independente de qual gateway processou o pagamento por trás.

Capacidades principais:

- Processamento de transações (crédito, débito, PIX, boleto) com roteamento automático;
- Motor antifraude com score 0–100 baseado em sinais compostos;
- Estorno (total) de transações aprovadas;
- Conciliação financeira por subadquirente e período;
- Webhooks de saída (eventos da plataforma para os sistemas do cliente) e de entrada (eventos recebidos de um subadquirente real — hoje, Necta Multi-Pay);
- Dashboard web ( [/dashboard](#) ) para operação e monitoramento.

## 2. Arquitetura & Stack

### 2.1 Stack tecnológica

| Camada          | Tecnologia                                                                                                                 |
|-----------------|----------------------------------------------------------------------------------------------------------------------------|
| Framework web   | FastAPI 0.111 (ASGI, assíncrono)                                                                                           |
| ORM / Banco     | SQLAlchemy 2.0 (async) — PostgreSQL (Neon) em produção, SQLite em desenvolvimento/testes                                   |
| Migrações       | Alembic                                                                                                                    |
| Autenticação    | JWT (python-jose) + 2FA TOTP (pyotp) + bcrypt (passlib)                                                                    |
| Filas / Cache   | Redis + Celery (worker definido, tarefas assíncronas)                                                                      |
| Cliente HTTP    | httpx (async) — usado nas integrações reais de subadquirente                                                               |
| Observabilidade | structlog, Sentry, Prometheus (via prometheus-fastapi-instrumentator, endpoint <code>/metrics</code> )                     |
| Testes          | pytest + pytest-asyncio (modo <code>auto</code> ), httpx.AsyncClient/ASGITransport, httpx.MockTransport para mocks de rede |
| Hospedagem      | Render (free tier) — domínio <code>complyroute.com.br</code>                                                               |

### 2.2 Estrutura de pastas

```
app/  
├─ api/v1/endpoints/      # routers FastAPI (um arquivo por recurso)  
├─ core/                  # config, security (JWT/hash/HMAC), exceptions, logging  
├─ db/                    # session, base declarativa, migrations (alembic)  
├─ models/                # modelos ORM (SQLAlchemy)  
├─ schemas/               # schemas Pydantic (request/response)  
├─ services/  
│   ├─ routing_engine.py  # motor de roteamento (12 regras)  
│   ├─ fraud_engine.py    # motor antifraude  
│   └─ acquirers/         # clientes reais de subadquirente (hoje: Necta)  
├─ workers/               # celery_app.py  
└─ static/dashboard.html  # dashboard operacional (SPA estática)  
scripts/                  # seed.py, necta_smoke_test.py  
tests/  
├─ unit/                  # motor de roteamento, cliente Necta  
└─ integration/           # um arquivo por router, contra a app real (ASGITransport)  
.github/workflows/        # keep-alive.yml (mantém o Render acordado)
```

### 2.3 Middleware de autenticação global

Um middleware HTTP único ( `main.py` ) protege **todas** as rotas por padrão, exigindo `Authorization: Bearer <jwt>` válido. As exceções explícitas são:

- Caminhos públicos fixos: `/`, `/health`, `/docs`, `/redoc`, `/openapi.json`, `/metrics`, `/dashboard`;

- `POST /api/v1/auth/login` e `POST /api/v1/auth/refresh` ;
- `POST /api/v1/acquirers/necta/webhook` — autenticado por assinatura Svix, não por JWT (é a Necta chamando o ComplyRoute, não um usuário).

Qualquer novo endpoint público (chamado por um sistema externo, não por um usuário logado) precisa ser adicionado explicitamente a essa lista em `main.py` — do contrário recebe `401` antes mesmo de chegar no router.

### 3. Autenticação & Autorização

#### 3.1 Papéis (roles)

| Papel     | Uso pretendido                                              |
|-----------|-------------------------------------------------------------|
| admin     | Acesso total                                                |
| financial | Operações financeiras (transações, conciliação, estornos)   |
| technical | Configuração técnica (roteamento, webhooks, subadquirentes) |
| readonly  | Somente leitura                                             |

Os papéis existem no modelo `User` e são embutidos no JWT ( `role` no payload), mas a autorização por papel **não é verificada** em nenhum endpoint hoje — o middleware apenas exige um JWT válido, independente do papel. Ver seção 15.

#### 3.2 Fluxo de login

| Método | Rota                 | Descrição                                                                                                                                  |
|--------|----------------------|--------------------------------------------------------------------------------------------------------------------------------------------|
| POST   | /api/v1/auth/login   | E-mail + senha. Se o usuário tiver 2FA ativo e <code>totp_code</code> não for enviado, retorna <code>requires_2fa: true</code> sem tokens. |
| POST   | /api/v1/auth/refresh | Troca um refresh token válido por um novo access token.                                                                                    |
| POST   | /api/v1/auth/logout  | Placeholder — em produção deve colocar o token numa blocklist Redis (não implementado).                                                    |

Access token: expira em `JWT_ACCESS_TOKEN_EXPIRE_MINUTES` (default 30min). Refresh token: `JWT_REFRESH_TOKEN_EXPIRE_DAYS` (default 7 dias). Algoritmo `HS256` , segredo em `SECRET_KEY` .

#### 3.3 Chaves de API

O modelo `APIKey` existe (prefixo `sk_live_ / sk_test_` , hash SHA-256, escopos `full / readonly / sandbox` ), e o dashboard tem um painel "API Keys" — mas **não há endpoint de API para gerenciá-las** hoje. É um recurso modelado no banco, ainda não exposto.

## 4. Motor de Roteamento

Implementado em `app/services/routing_engine.py` como uma lista **fixa, no código** de 12 regras avaliadas em ordem de prioridade — não é lida da tabela `routing_rules` no banco (essa tabela e o endpoint `GET /api/v1/routing/rules` hoje só espelham/listam, não alimentam o motor).

### 4.1 As 12 regras (em ordem de avaliação)

| #  | Regra                                                           | Ação           |
|----|-----------------------------------------------------------------|----------------|
| 1  | PIX → Rede                                                      | route          |
| 2  | Boleto → bloquear SafraPay (não suporta boleto)                 | block_acquirer |
| 3  | Débito → bloquear OrendaPay (não suporta débito)                | block_acquirer |
| 4  | Crédito acima de R\$ 10.000 → Cielo                             | route          |
| 5  | Boleto → Asaas                                                  | route          |
| 6  | PIX → Rede (redundante com a #1)                                | route          |
| 7  | Débito → Cielo                                                  | route          |
| 8  | Parcelas $\geq 12x$ → Stone                                     | route          |
| 9  | BIN internacional → PagSeguro ( <i>desativada por padrão</i> )  | route          |
| 10 | Score antifraude $< 50$ → bloquear                              | block          |
| 11 | Timeout no adquirente primário → GetNet (fallback)              | route          |
| 12 | Retry automático em timeout (até <code>TXN_MAX_RETRIES</code> ) | retry          |

A regra 10 (antifraude) é sempre avaliada **primeiro**, com prioridade máxima, antes de qualquer outra — inclusive antes do override manual descrito abaixo. Se nenhuma regra rotear a transação, o fallback padrão é `DEFAULT_ACQUIRER = "cielo"`.

### 4.2 Override manual ( `metadata.force_acquirer` )

Para validar integrações novas sem alterar o roteamento padrão de ninguém, uma transação pode enviar `"metadata": {"force_acquirer": "necta"}` e pular direto para aquele subadquirente. Duas restrições de segurança:

- Avaliado **depois** do bloqueio antifraude — não serve para contornar o antifraude;
- Só funciona quando `APP_ENV != "production"` — não é uma feature para o chamador da API escolher livremente o subadquirente em produção.

### 4.3 Simulação

`POST /api/v1/routing/simulate` avalia as regras sem criar uma transação — retorna o subadquirente selecionado e a trilha completa de decisão (cada regra avaliada, se casou e qual ação tomou). Útil para depurar por que uma transação foi (ou seria) roteada para determinado subadquirente.

## 5. Motor Antifraude

Implementado em `app/services/fraud_engine.py`. Score final = `100 - média ponderada das penalidades` dos cinco sinais abaixo. Score < 50 bloqueia a transação.

| Sinal                                          | Peso | Status de implementação                                               |
|------------------------------------------------|------|-----------------------------------------------------------------------|
| Velocidade de IP                               | 30%  | Placeholder — sempre 0 (produção: janela deslizante no Redis)         |
| Device fingerprint                             | 20%  | Penalidade leve (10) só quando ausente; sem checagem real de reuso    |
| BIN em lista negra                             | 25%  | Placeholder — sempre 0 (produção: tabela de BINs bloqueados)          |
| Divergência geográfica (país do cartão vs. IP) | 15%  | Placeholder — sempre 0 (produção: geolocalização de IP)               |
| Valor anômalo para o método                    | 10%  | Implementado: PIX > R\$20.000 e crédito à vista > R\$30.000 penalizam |

A maioria dos sinais são placeholders documentados no próprio código-fonte. O motor está com a arquitetura pronta (interface, pesos, composição), mas os sinais de maior peso (velocidade de IP, BIN) ainda não têm implementação real.

## 6. Transações

| Método | Rota                             | Descrição                                                                                            |
|--------|----------------------------------|------------------------------------------------------------------------------------------------------|
| POST   | /api/v1/transactions             | Cria e processa uma transação: antifraude → roteamento → subadquirente → persistência.               |
| GET    | /api/v1/transactions             | Lista com paginação e filtros ( <code>status</code> , <code>method</code> , <code>acquirer</code> ). |
| GET    | /api/v1/transactions/{id}        | Detalhe de uma transação.                                                                            |
| POST   | /api/v1/transactions/{id}/refund | Estorna uma transação aprovada (total).                                                              |

### 6.1 Ciclo de vida do status

`pending` → `processing` → `approved` | `declined` , e a partir de `approved` : `refunded` ou `chargeback` .

### 6.2 Envio ao subadquirente

Após o roteamento decidir o subadquirente, o endpoint consulta `AcquirerClientFactory.get(nome)` :

- Se existir um cliente real configurado (hoje, apenas `"necta"` , e só quando `NECTA_CLIENT_SECRET` / `NECTA_SECRET_KEY` estão definidos) → chama a API real do gateway;
- Caso contrário (todos os demais subadquirentes hoje) → **resposta simulada** ( `approved` , código de autorização e NSU gerados localmente). Isso é intencional e pré-existe à integração Necta: nenhum outro subadquirente tem cliente HTTP real implementado ainda.

Uma falha do cliente real ( `AcquirerError` ) persiste a tentativa como `declined` com o motivo do erro e propaga `502` / `504` ao chamador.

## 7. Subadquirentes

| Nome                   | Prioridade (seed) | Débito | PIX | Boleto | Integração real           |
|------------------------|-------------------|--------|-----|--------|---------------------------|
| Cielo                  | 1 (primário)      | ✓      | ✓   | ✓      | Simulada                  |
| Rede                   | 2                 | ✓      | ✓   | ✓      | Simulada                  |
| Stone                  | 3                 | ✓      | ✓   | ✓      | Simulada                  |
| PagSeguro              | 4                 | ✓      | ✓   | ✓      | Simulada                  |
| GetNet                 | 5 (fallback)      | ✓      | ✓   | ✓      | Simulada                  |
| SafraPay               | 6                 | ✓      | ✓   | ✗      | Simulada                  |
| Zoop                   | 7                 | ✓      | ✓   | ✓      | Simulada                  |
| Asaas                  | 8                 | ✓      | ✓   | ✓      | Simulada                  |
| OrendaPay              | 9                 | ✗      | ✓   | ✓      | Simulada                  |
| <b>Necta Multi-Pay</b> | 10                | ✗      | ✓   | ✓      | <b>Real (ver seção 8)</b> |

| Método | Rota                            | Descrição                              |
|--------|---------------------------------|----------------------------------------|
| GET    | /api/v1/acquirers               | Lista todos, ordenados por prioridade. |
| GET    | /api/v1/acquirers/{name}/status | Status detalhado de um subadquirente.  |

## 8. Integração — Necta Multi-Pay

Única integração com um subadquirente real hoje. Cliente em `app/services/acquirers/necta.py`, contra a API pública documentada em `doc.nectaco.com.br` (OpenAPI 3.1, servida em `/openapi/v1`).

### 8.1 Autenticação

Máquina-a-máquina: `POST /auth` com `{clientSecret, secretKey}` (gerados em `POST /api-tokens` no painel Necta) retorna um JWT usado em `Authorization: Bearer`. O `NectaClient` cacheia o token por 25 minutos e reautentica automaticamente uma vez em caso de `401`.

### 8.2 Criação de venda (um passo por método)

| Método interno      | Endpoint Necta                                   |
|---------------------|--------------------------------------------------|
| <code>pix</code>    | <code>POST /sales/pix</code>                     |
| <code>credit</code> | <code>POST /sales/credit-card</code>             |
| <code>boleto</code> | <code>POST /sales/bank-slip</code>               |
| <code>debit</code>  | Não suportado — lança <code>AcquirerError</code> |

O comprador ( `buyer` ) exige nome, documento, e-mail, telefone e endereço completo. Como o schema `CustomerData` do ComplyRoute só exigia nome/documento/e-mail, foram adicionados campos opcionais `phone_number` e `address`. Quando ausentes, o cliente registra um aviso e envia um endereço placeholder — aceitável para teste, não recomendado para tráfego real (ver seção 15).

### 8.3 Mapeamento de status

| Status Necta                                                | Status ComplyRoute      |
|-------------------------------------------------------------|-------------------------|
| <code>pending, processing, scheduled, pre_authorized</code> | <code>processing</code> |
| <code>paid</code>                                           | <code>approved</code>   |
| <code>refunded, partially_refunded</code>                   | <code>refunded</code>   |
| <code>canceled, denied, refused, expired</code>             | <code>declined</code>   |

### 8.4 Estorno

`POST /sales/{uuid}/void` — sem corpo é estorno total; com `{"amount": N}` seria parcial (Necta só suporta parcial em alguns gateways; o ComplyRoute hoje só envia estorno total pelo endpoint `/transactions/{id}/refund`).

### 8.5 Webhook de entrada

`POST /api/v1/acquirers/necta/webhook` recebe `sale.paid`, `sale.refunded` e `sale.failed` da Necta e atualiza a transação correspondente (localizada por `acquirer_txn_id`). Isento do middleware de JWT (seção 2.3) — autenticado por assinatura no padrão **Svix**:

```
svix-id: msg_...
svix-timestamp: <unix ts>
svix-signature: v1,<base64 hmac-sha256>

segredo: NECTA_WEBHOOK_SECRET (formato whsec_<base64>)
conteúdo assinado: "{svix-id}.{svix-timestamp}.{corpo bruto}"
tolerância de replay: 300s
```

A URL `https://complyroute.com.br/api/v1/acquirers/necta/webhook` precisa ser cadastrada manualmente no painel da Necta (evento por evento, ou "todos") para que os eventos cheguem — isso é um passo operacional, não uma configuração de código.

## 8.6 Ferramentas de validação

- `tests/unit/test_necta_client.py` — 12 testes cobrindo autenticação/cache, reauth em 401, montagem de payload por método, mapeamento de status e verificação de assinatura Svix, usando `httpx.MockTransport` (sem rede real);
- `tests/integration/test_acquirers.py` — testes do endpoint de webhook (assinatura válida, inválida, venda desconhecida);
- `scripts/necta_smoke_test.py` — chama o `NectaClient` direto contra a API real (sandbox/produção conforme `.env`), fora do resto da aplicação: `python scripts/necta_smoke_test.py [--amount N] [--refund UUID]`.

## 9. Webhooks

---

### 9.1 Saída — eventos do ComplyRoute para o cliente

| Método | Rota                  | Descrição                            |
|--------|-----------------------|--------------------------------------|
| GET    | /api/v1/webhooks      | Lista URLs cadastradas.              |
| POST   | /api/v1/webhooks      | Cadastra uma URL + lista de eventos. |
| DELETE | /api/v1/webhooks/{id} | Remove.                              |

Modelos `Webhook` (URL, eventos, segredo HMAC, taxa de sucesso) e `WebhookDelivery` (histórico de entregas) existem; a assinatura HMAC-SHA256 de saída já tem utilitários prontos em `app/core/security.py` ( `sign_webhook_payload` / `verify_webhook_signature` ), mas o disparo assíncrono real das entregas (via Celery, mencionado como TODO em `transactions.py` ) ainda não está implementado.

### 9.2 Entrada — eventos recebidos de um subadquirente real

Ver seção 8.5. Hoje existe apenas para a Necta.

# 10. Conciliação

| Método | Rota                           | Descrição                                                                                                                            |
|--------|--------------------------------|--------------------------------------------------------------------------------------------------------------------------------------|
| GET    | /api/v1/reconciliation/summary | Resumo por subadquirente num período ( date_from / date_to , default hoje/UTC): total de transações, volume bruto e volume aprovado. |

# 11. Configuração & Variáveis de Ambiente

Todas as configurações vêm de `app/core/config.py` (Pydantic Settings), carregadas de `.env`. Veja `.env.example` para a lista completa com comentários. Principais grupos:

| Grupo                      | Variáveis                                                                                                                                                                                                              |
|----------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| App                        | <code>APP_NAME</code> , <code>APP_VERSION</code> , <code>APP_ENV</code> , <code>DEBUG</code> , <code>SECRET_KEY</code> , <code>ALLOWED_ORIGINS</code> , <code>ALLOWED_HOSTS</code>                                     |
| Banco                      | <code>DATABASE_URL</code> (async), <code>DATABASE_URL_SYNC</code> (migrações), pool                                                                                                                                    |
| Redis/Celery               | <code>REDIS_URL</code> , <code>CELERY_BROKER_URL</code> , <code>CELERY_RESULT_BACKEND</code>                                                                                                                           |
| JWT                        | <code>JWT_ALGORITHM</code> , <code>JWT_ACCESS_TOKEN_EXPIRE_MINUTES</code> , <code>JWT_REFRESH_TOKEN_EXPIRE_DAYS</code>                                                                                                 |
| Webhook (saída)            | <code>WEBHOOK_SECRET</code>                                                                                                                                                                                            |
| Antifraude / Limites       | <code>FRAUD_SCORE_MIN_*</code> , <code>TXN_MIN_AMOUNT</code> , <code>TXN_MAX_AMOUNT</code> , <code>TXN_MAX_INSTALLMENTS</code> , <code>TXN_TIMEOUT_MS</code> , <code>TXN_MAX_RETRIES</code>                            |
| Subadquirentes (simulados) | <code>CIELO_*</code> , <code>REDE_*</code> , <code>STONE_*</code> , <code>PAGSEGURO_*</code> , <code>GETNET_*</code> , <code>SAFRAPAY_*</code> , <code>ZOOP_*</code> , <code>ASAAS_*</code> , <code>ORENDAPAY_*</code> |
| Necta (integração real)    | <code>NECTA_CLIENT_SECRET</code> , <code>NECTA_SECRET_KEY</code> , <code>NECTA_API_URL</code> , <code>NECTA_WEBHOOK_SECRET</code>                                                                                      |
| Observabilidade            | <code>SENTRY_DSN</code> , <code>LOG_LEVEL</code>                                                                                                                                                                       |

## 12. Testes Automatizados

```
.venv\Scripts\python.exe -m pytest -q          # suíte completa
.venv\Scripts\python.exe -m pytest tests/unit -q
.venv\Scripts\python.exe -m pytest tests/integration -q
```

Configuração em `pyproject.toml` ( `asyncio_mode = "auto"` — testes `async def` não precisam de decorator). Fixtures compartilhadas em `tests/conftest.py` : banco SQLite em memória isolado por teste e um `AsyncClient` contra a app real via `ASGITransport` (sem subir servidor). Chamadas de rede para subadquirentes são sempre mockadas com `httpx.MockTransport` — nenhum teste depende de internet ou de credenciais reais.

### 12.1 Cobertura por área

| Arquivo                                        | Cobre                                                                                                                                              |
|------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>tests/unit/test_routing_engine.py</code> | As 12 regras, bloqueio antifraude, override <code>force_acquirer</code> (inclusive seu bloqueio em produção e sua não-interferência no antifraude) |
| <code>tests/unit/test_necta_client.py</code>   | Cliente Necta isolado (auth, retry, payloads, assinatura de webhook)                                                                               |
| <code>tests/integration/test_*.py</code>       | Um arquivo por router: auth, transactions, acquirers, webhooks, users, routing, settings, reconciliation                                           |

## 13. Deploy & Infraestrutura

---

Hospedado no **Render** (instância free) sob o domínio `complyroute.com.br`. Não há `render.yaml` no repositório — a configuração do serviço é feita diretamente no dashboard da Render.

### 13.1 Cold start e o workflow de keep-alive

Instâncias free do Render entram em modo de espera após ~15 minutos sem tráfego, e a próxima requisição sofre um "cold start" (tela de "Service waking up" por alguns segundos). Para reduzir a frequência disso, `.github/workflows/keep-alive.yml` faz um `GET /health` a cada 10 minutos via GitHub Actions.

Isso é um paliativo, não uma garantia: (1) o GitHub Actions pode atrasar execuções agendadas, especialmente nos primeiros minutos após o workflow ser criado ou em horários de pico; (2) workflows agendados são **desativados automaticamente pelo GitHub após 60 dias sem nenhum commit** no repositório. A eliminação completa do cold start exige upgrade para uma instância paga do Render.

## 14. Histórico de Versões

| Versão | Data       | Mudança                                                                                                                                                                                  |
|--------|------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1.0.0  | 16/07/2026 | Commit inicial — backend do gateway ComplyRoute + dashboard operacional                                                                                                                  |
| 1.0.1  | 16/07/2026 | Correção no parsing de <code>ALLOWED_ORIGINS</code> / <code>ALLOWED_HOSTS</code>                                                                                                         |
| 1.0.2  | 16/07/2026 | Fixação de <code>bcrypt==3.2.2</code> (incompatibilidade com passlib quebrava o login)                                                                                                   |
| 1.1.0  | 16/07/2026 | Dashboard responsivo para mobile/iPhone Safari                                                                                                                                           |
| 1.1.1  | 16/07/2026 | Suíte de testes de integração para todos os routers da API; 3 bugs corrigidos no processo                                                                                                |
| 1.1.2  | 17/07/2026 | Gráfico de transações por hora redimensionado (widget compacto)                                                                                                                          |
| 1.1.3  | 17/07/2026 | Gráfico de transações por hora no padrão visual Uber                                                                                                                                     |
| 1.2.0  | 04/08/2026 | Redesign do gráfico por hora e da distribuição por adquirente (padrão Uber)                                                                                                              |
| 1.2.1  | 11/08/2026 | Workflow agendado para manter o serviço Render acordado                                                                                                                                  |
| 1.3.0  | 11/08/2026 | <b>Integração real com a Necta Multi-Pay</b> como subadquirente (autenticação M2M, criação de vendas PIX/crédito/boleto, estorno, webhook de entrada com verificação de assinatura Svix) |
| 1.3.1  | 11/08/2026 | Ferramentas de validação da integração Necta: script de smoke test e override <code>force_acquirer</code> para testes ponta a ponta                                                      |

## 15. Pendências Conhecidas

---

Lista honesta do que ainda falta, para orientar o próximo trabalho — nada aqui é urgente por si só, mas cada item foi observado diretamente no código durante esta documentação.

### Roteamento

- As 12 regras são fixas no código; a tabela `routing_rules` e o endpoint `GET /routing/rules` não as alimentam de fato — não há hoje como um operador mudar o roteamento sem um deploy.
- As flags `Acquirer.supports_debit/pix/boleto` existem no banco mas não são consultadas pelo motor de roteamento.

### Autorização

- Os papéis ( `admin` / `financial` / `technical` / `readonly` ) estão no JWT mas nenhum endpoint valida o papel do usuário — qualquer usuário autenticado pode chamar qualquer rota.

### Antifraude

- Os sinais de maior peso (velocidade de IP — 30%, BIN em lista negra — 25%, divergência geográfica — 15%) são placeholders que sempre retornam 0 — o score hoje é, na prática, dominado pelo sinal de valor anômalo (10%) e pela ausência de device fingerprint (20%).

### Integração Necta

- Sem suporte a 3DS para cartão de crédito (a API da Necta oferece o fluxo completo);
- Sem pré-autorização/captura em dois passos;
- Cartão trafega como PAN bruto — a Necta também oferece tokenização client-side ( `networkToken` / `cardTokenId` ) que reduziria o escopo PCI;
- Sem job periódico de conciliação/reconsulta para vendas presas em `processing` caso a entrega de um webhook falhe — hoje depende inteiramente do webhook chegar;
- Quando o comprador não informa telefone/endereço completos, o cliente envia dados placeholder em vez de rejeitar a venda — aceitável para teste, questionável para produção.

### Webhooks de saída

- Modelo e assinatura HMAC já existem, mas o disparo assíncrono real (via Celery) ainda é um TODO — hoje nada é efetivamente entregue aos clientes cadastrados.

### Chaves de API

- Modelo `APIKey` existe e o dashboard tem um painel para isso, mas não há endpoint de API para criar/listar/revogar chaves.

### Deploy

- Sem `render.yaml` — a configuração do serviço vive só no dashboard da Render, não versionada;
- O workflow de keep-alive reduz mas não elimina o cold start (ver seção 13.1).